

6. Process

6.1 Project Organisation and Tools

The project is split across three Git repositories, each with its own Jenkins CI/CD pipeline deploying to AWS EC2 via Docker:

Repository	Role
flask-app	REST API (Flask/Gunicorn), JWT auth, ML predictions, journey planning, AI chat. Owns the MySQL schema via Flask-Migrate.
react-app	Single-page frontend (React 19, TypeScript, Vite 7, Tailwind CSS 4). Served by nginx in production.
scraper	Long-running Python process: polls JCDecaux stations and OpenWeatherMap forecasts, writing to the shared database.

Tooling: Git feature-branch workflow • Jenkins (Kubernetes agent) • Docker • AWS EC2 • pytest ($\approx 97\%$ coverage) • ESLint / TypeScript • Discord (team communication).

Integration contract: Both flask-app and scraper share one DATABASE_URL. Flask-Migrate migrations must run first. In development, Vite proxies /api to Flask on port 5000; in production a reverse proxy handles the same routing.

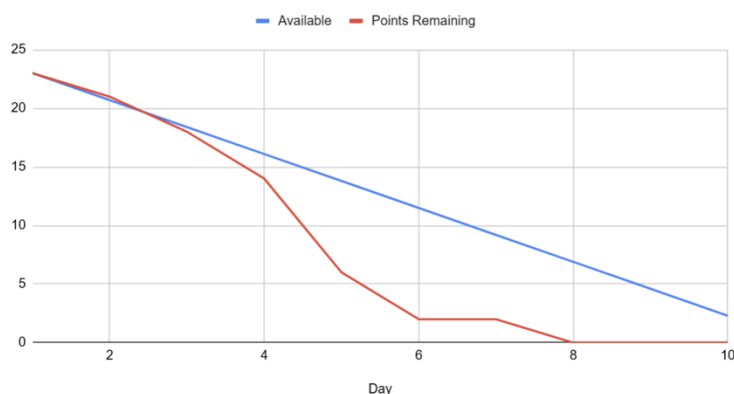
6.2 Sprint 1 — Requirements Engineering & Data Collection

Implemented Features and Design Decisions

The team conducted user interviews to validate three personas (Student, Professional, Tourist), produced Figma UI mockups, and defined user stories. On the technical side, a MySQL database schema was established and scraping scripts for the JCDecaux and OpenWeatherMap APIs were written and deployed. A 5-minute scrape interval was chosen as the design default to balance data freshness against API rate limits. The scraper ran stably throughout the sprint, consistently populating the database and providing a reliable foundation for subsequent development.

Burndown Chart

Available and Points Remaining



The Actual Remaining Effort line crossed below the Ideal Trend line on Day 4 and reached zero two days before the deadline, indicating the team worked ahead of schedule. The surplus time was used for backlog refinement and feasibility research on the Smart Journey Planner feature.

Sprint Review

- Presented Figma mockups aligned to persona pain-points (tourist navigation, weather prediction, map interactivity).
- Demonstrated live MySQL tables populated by the running scrapers.
- Opened a scope discussion on the Journey Planner feature; decision deferred to Sprint 2 planning.

Sprint Retrospective

What went well: Discord channel established for daily updates. User interviews successfully validated all three personas, directly shaping the decision to prioritise map interactivity. Scraping scripts delivered ahead of schedule and ran stably.

What could be improved: Some members were unfamiliar with API requests and local MySQL configuration, causing early delays. Peer support will be increased in future sprints to resolve technical blockers faster.

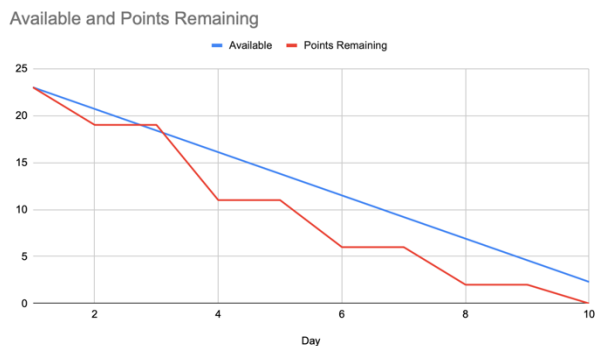
For next sprint: Set up the Flask application and connect it to the database; get scraped data displaying on a live map; finalise the Journey Planner feasibility study.

6.3 Sprint 2 — Backend Integration & Deployment Pipeline

Implemented Features and Design Decisions

The Flask application was built out with Weather and Station Read APIs returning properly formatted JSON from the shared MySQL database. JWT-based user authentication was integrated with secure password hashing. A Jenkins CI/CD pipeline was configured with a Kubernetes agent, automating Docker builds and EC2 deployments on every push to main. A Git feature-branch workflow was adopted team-wide. Crucially, the Journey Planner backend endpoint was delivered, closing the open scope question from Sprint 1, though it required significant refactoring to handle edge cases and correct route-calculation logic.

Burndown Chart



The chart shows a staircase pattern reflecting the mandated two-day update cadence. Despite this, the Actual Remaining Effort line dropped below the Ideal Trend by Day 4 and stayed beneath it for the rest of the sprint, confirming strong team velocity. All goals were completed by Day 10.

Sprint Review

- Demonstrated Weather and Station APIs returning live JSON data.
- Showcased JWT login flow and hashed password storage.
- Live CI/CD demo: a code push triggered a Jenkins build and automatic EC2 deployment.
- Confirmed Journey Planner inclusion and basic backend functionality.

Sprint Retrospective

What went well: The Git feature-branch workflow and Jenkins pipeline accelerated testing and removed manual deployment effort. Resolving the scope question early enabled delivery of the complex Journey Planner and secure login features.

What could be improved: API endpoints lacked documentation at the outset, forcing frontend developers to read backend source code to find correct routes. The Journey Planner also required substantial refactoring for edge-case handling.

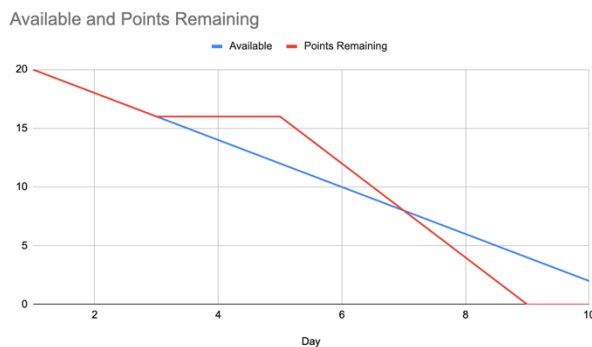
For next sprint: Build React UIs for the Journey Planner and Login; migrate the database to Amazon RDS; produce API documentation to streamline frontend–backend integration.

6.4 Sprint 3 — Additional Features & Frontend Implementation

Implemented Features and Design Decisions

Three AI chat features were implemented using the Langchain framework and the Aliyun Qwen API, with chat history persisted in new database tables and displayed on login. The React frontend was updated to show available bike and stand counts on station hover, and data is now pre-fetched on page load to eliminate display latency. The database was migrated from a local instance to Amazon RDS to overcome EC2 memory constraints and improve query performance. The CI/CD pipeline was verified end-to-end with Docker containerisation confirming a stable, reproducible deployment process.

Burndown Chart



The burndown chart was not explicitly described in the sprint records; the team confirmed all sprint goals were completed within the sprint window.

Sprint Review

- Demonstrated AI chat with history storage and LLM memory via new database tables.
- Confirmed Flask backend completeness through Postman testing.
- Showed hover-based bike/stand data on the map with zero page-load latency.
- Verified Jenkins CI/CD and Docker containerisation for both development and production.

Sprint Retrospective

What went well: Core features are working. Frontend and backend integration was smooth and error-free. Development and production environments are correctly configured, and the code-review workflow is operating well.

What could be improved: UI/UX design and colour scheme need further refinement. API keys risk exposure in the browser console, a security concern for production. AI chat output can be interrupted if the user navigates away mid-stream.

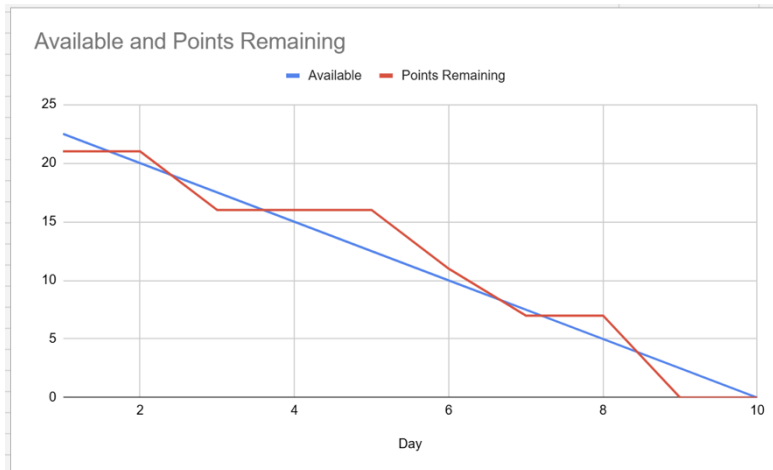
For next sprint: Improve UI/UX and colour scheme; fix AI chat response persistence; implement ML-based availability predictions; add unit, integration, and end-to-end tests.

6.5 Sprint 4 — Machine Learning Integration & System Finalisation

Implemented Features and Design Decisions

A Random Forest model was trained on historical station and weather data, serialised as a .pkl file, and integrated into the Flask backend via a prediction service that preloads the model at startup. The React frontend was updated to fetch and render predictions dynamically, with a brief cold-start latency on the first request acknowledged and documented. UI styling was refined for consistency across all pages. Backend test coverage reached 97% with pytest, and the CI/CD pipeline was updated to download ML artefacts from Hugging Face Hub during the build stage.

Burndown Chart



Points Remaining stayed slightly above the Ideal Trend line during Days 3–5 as the team focused on ML fine-tuning and API formatting issues. A steep drop on Day 7 restored velocity, and all tasks were completed on Day 9, one day before the deadline.

Sprint Review

- Demonstrated the fully integrated web application: React frontend → Flask backend → ML predictions rendered live.
- Explained cold-start latency on first prediction request and confirmed fast performance on subsequent requests.
- Highlighted 97% unit test coverage as evidence of system reliability.
- Agreed final phase will focus on code refactoring, UI/UX polish, and report compilation.

Sprint Retrospective

What went well: The full ML pipeline — data cleaning, feature selection, model training, export, and Flask integration — was delivered successfully. React UI components were implemented and styled consistently. 97% test coverage was achieved, confirming system robustness.

What could be improved: Algorithm comparison and hyperparameter tuning took longer than planned. Minor API formatting issues during frontend–backend integration required extra debugging effort. Cold-start prediction latency was identified as a known limitation.

Final conclusion: All core development tasks are complete. Data scraping, the Flask backend, the React frontend, and ML predictions have been successfully integrated into a fully functional web application.